

Clinical Cloud vs Edge Pipeline Benchmark

Detailed explanation of the patient dataset, local edge pipeline, Google Cloud Run deployment, transmitted information, security modes, executed benchmark pipelines, metrics, and hospital dashboard.

```
Repository: cloud-edge-pipeline-benchmark
Dataset: data/patients.json
Google Cloud project: benchmark-edge-cloud
Google Cloud region: europe-west8
Cloud Run service URL: https://benchmark-cloud-api-x7byt6caoq-oc.a.run.app
Dashboard: http://localhost:8080
Local benchmark: docker compose run --rm benchmark
Hybrid GCP benchmark: see Section 11 commands
```

1. What This Benchmark Demonstrates

The project compares two ways of processing hospital monitoring data. The cloud pipeline sends the complete patient record to a centralized API and performs every clinical processing step there. The edge pipeline processes the same record near the ward or bedside and creates a local early-warning alert. Cloud synchronization is not included in the measured edge latency, so the benchmark is a direct edge vs cloud comparison. The project also supports a hybrid benchmark where the cloud API is deployed on Google Cloud Run in Europe while the edge service and benchmark runner stay local.

Important: the patients are synthetic. They are realistic enough for benchmarking and dashboard design, but they are not real patients and must not be interpreted as medical advice.

2. Patient Dataset

The benchmark uses 20 synthetic patient records. Each record contains demographics, location, diagnosis, comorbidities, medications, and three recent vital-sign samples.

```
patient_id, name, age, sex
ward, bed
primary_diagnosis
comorbidities[]
medications[]
vitals[]:
  timestamp
  heart_rate
  systolic_bp
  respiratory_rate
  spo2
  temperature
  glucose
```

Dataset snapshot

Patient	Ward / Bed	Diagnosis	Latest Vitals
P-0001 Anna Rossi (78F)	Cardiology C-12	Heart failure exacerbation	HR 108 SBP 122 RR 25 SpO2 92% T 37.0 Glu 118
P-0002 Marco Bianchi (64M)	Internal Medicine M-04	Community acquired pneumonia	HR 121 SBP 108 RR 30 SpO2 88% T 38.7 Glu 191
P-0003 Giulia Conti (42F)	Neurology N-09	Migraine observation	HR 76 SBP 120 RR 16 SpO2 99% T 36.5 Glu 91
P-0004 Luca Ferri (70M)	Emergency E-02	Suspected sepsis	HR 135 SBP 84 RR 34 SpO2 89% T 39.5 Glu 155
P-0005 Sara Greco (55F)	Surgery S-18	Post-operative monitoring	HR 95 SBP 121 RR 20 SpO2 96% T 37.5 Glu 111

Patient	Ward / Bed	Diagnosis	Latest Vitals
P-0006 Davide Romano (83M)	Geriatrics G-07	Dehydration and delirium	HR 115 SBP 96 RR 25 SpO2 94% T 38.0 Glu 218
P-0007 Elena Gallo (35F)	Obstetrics O-03	Postpartum observation	HR 86 SBP 112 RR 17 SpO2 98% T 37.0 Glu 92
P-0008 Paolo Ricci (59M)	Cardiology C-04	Acute coronary syndrome observation	HR 107 SBP 140 RR 22 SpO2 95% T 36.8 Glu 131
P-0009 Martina Riva (47F)	Oncology ON-11	Chemotherapy fever	HR 123 SBP 98 RR 26 SpO2 92% T 39.1 Glu 127
P-0010 Roberto Serra (68M)	Pulmonology P-15	COPD exacerbation	HR 114 SBP 126 RR 30 SpO2 86% T 37.8 Glu 140
P-0011 Chiara Lombardi (29F)	Emergency E-08	Syncope observation	HR 72 SBP 110 RR 15 SpO2 99% T 36.5 Glu 88
P-0012 Francesco Moretti (73M)	Nephrology K-06	Acute kidney injury	HR 101 SBP 94 RR 23 SpO2 95% T 37.5 Glu 180
P-0013 Valentina Neri (51F)	Internal Medicine M-14	Anemia workup	HR 90 SBP 110 RR 18 SpO2 97% T 36.8 Glu 101
P-0014 Stefano Costa (61M)	ICU Stepdown I-05	Post-sepsis recovery	HR 109 SBP 106 RR 26 SpO2 92% T 38.1 Glu 154
P-0015 Noemi De Luca (24F)	Surgery S-03	Appendectomy observation	HR 82 SBP 116 RR 16 SpO2 98% T 37.0 Glu 93
P-0016 Giorgio Fontana (80M)	Stroke Unit ST-10	Ischemic stroke monitoring	HR 102 SBP 154 RR 22 SpO2 95% T 36.8 Glu 145
P-0017 Irene Marchetti (66F)	Endocrinology D-02	Hyperglycemia	HR 95 SBP 132 RR 19 SpO2 97% T 36.8 Glu 280
P-0018 Alessio Vitale (57M)	Trauma T-06	Rib fracture monitoring	HR 108 SBP 118 RR 26 SpO2 92% T 37.3 Glu 108
P-0019 Federica Barbieri (72F)	Cardiology C-17	Atrial fibrillation rate control	HR 132 SBP 120 RR 23 SpO2 94% T 37.0 Glu 124
P-0020 Matteo Villa (39M)	Infectious Diseases ID-01	Viral syndrome	HR 96 SBP 116 RR 19 SpO2 97% T 38.3 Glu 103

3. Cloud Pipeline in Detail

The cloud pipeline represents a centralized architecture. The complete record is transmitted to the cloud API. This maximizes central visibility but also sends the largest amount of clinical information over the network.

```
benchmark client
-> cloud-api /process
-> simulated network delay
-> schema validation
-> vital-sign normalization and range clamping
-> feature extraction on the full time series
-> vital-sign score
-> trend score
-> clinical context score
-> risk classification and triage recommendation
-> full-record storage
-> response to client
```

Information sent to cloud

- Full demographics: patient identifier, name, age, sex.
- Hospital context: ward and bed.
- Clinical context: diagnosis, comorbidities, medications.
- Complete vital-sign time series: heart rate, systolic blood pressure, respiratory rate, oxygen saturation, temperature, glucose.

Information stored by cloud

- The full patient record.
- The complete vital-sign time series.
- Extracted features: latest values, averages, trends.
- Risk score, risk level, component scores, alert flag, and recommended action.

4. Edge Pipeline in Detail

The edge pipeline represents processing close to the source. The edge receives the same raw record and performs early-warning logic locally. In this benchmark, cloud synchronization is disabled for edge scenarios, so the latency reflects the local edge pipeline only.

```
benchmark client
-> edge-api /process
-> simulated local network delay
-> schema validation
-> vital-sign range clamping
-> moving-average noise filter
-> local feature extraction
-> early-warning score
-> local alert generation
-> response to client
```

Information processed locally at the edge

- The complete patient record.
- The complete vital-sign time series.
- Filtered vital-sign samples.
- Local clinical features and risk score.

Information produced by the edge

```
patient_id
patient_hash pseudonymized from patient_id
```

```
ward
bed
latest_vitals
risk_score
risk_level
recommended_action
alert
```

In the direct comparison, this reduced payload is returned to the benchmark client but is not synchronized to the cloud. Therefore, edge scenarios report `payload_synced_kb` as 0 and `sync_ms` as 0.

5. Google Cloud Run Deployment

To make the benchmark closer to a real deployment, the cloud API container can be deployed on Google Cloud Run. In the current setup, the Google Cloud project is `benchmark-edge-cloud`, the selected European region is `eu-west-8`, and the deployed Cloud Run endpoint is `https://benchmark-cloud-api-x7byt6caq-oc.a.run.app`.

```
Google Cloud
Artifact Registry: europe-west8-docker.pkg.dev/benchmark-edge-cloud/benchmark/pipeline-api:latest
Cloud Run service: benchmark-cloud-api
Public HTTPS endpoint: https://benchmark-cloud-api-x7byt6caq-oc.a.run.app
Runtime role: cloud
Transport security label: platform_tls

Local machine / simulated hospital edge
edge-api
benchmark-runner
prometheus
grafana
hospital-dashboard
```

Cloud Run terminates public HTTPS at the Google-managed platform boundary. For this reason, the hybrid Google Cloud Run scenarios are labelled `platform_tls` rather than container-terminated mTLS. The local `cloud_tls` and `edge_tls` scenarios still use the repository's local mTLS certificates and remain useful for measuring container-level TLS overhead.

Hybrid Google Cloud execution path

```
benchmark-runner local
-> HTTPS request to Google Cloud Run /process for cloud scenarios

benchmark-runner local
-> local edge-api /process for edge scenarios

prometheus local
-> scrapes local edge-api /metrics
-> scrapes Google Cloud Run /metrics over HTTPS
```

- The cloud path includes real internet routing from the local machine to Google Cloud Run.
- The edge path keeps clinical preprocessing and risk scoring local and does not include cloud synchronization in the measured latency.
- Prometheus observes both local edge metrics and remote cloud metrics, while Grafana visualizes the collected time series.
- The hospital dashboard continues to read benchmark result files from the local results directory.

6. Clinical Processing Logic

The processing logic is implemented in `services/pipeline_api/app/clinical.py`.

Step	Cloud	Edge
Validation	Checks required fields and vital samples.	Same validation before local processing.
Cleaning	Clamps vital signs to plausible ranges.	Clamps vital signs to plausible ranges.
Noise filtering	Uses the raw time series for full central analysis.	Applies moving-average smoothing before scoring.
Feature extraction	Computes latest values, averages, and trends from the full time series.	Computes latest values, averages, and trends from filtered vitals.
Risk scoring	Vital score + trend score + clinical context score.	Same scoring logic on filtered local features.
Alerting	Cloud triage alert after central processing.	Immediate local alert before cloud synchronization.
Storage/sync	Stores full clinical payload.	No cloud sync in the measured edge pipeline.

Risk score components

- Vital score: heart rate, systolic pressure, respiratory rate, SpO2, temperature, glucose.
- Trend score: worsening heart rate, respiratory rate, SpO2, systolic pressure, and temperature across the samples.
- Context score: age, number of comorbidities, and high-risk diagnosis terms such as sepsis, pneumonia, COPD, heart failure, or stroke.
- Risk level: low, medium, high, or critical.

7. Executed Benchmark Pipelines

```
cloud
edge
cloud_simulated_secure
edge_simulated_secure
cloud_tls
edge_tls
```

The benchmark compares the user-facing `/process` pipeline for cloud and edge. Edge scenarios do not call the `cloud/sync` endpoint during measurement, so the comparison is direct.

Local Docker benchmark scenarios

Scenario	Processing	Transport	Purpose
cloud	Full central processing	HTTP	Baseline centralized pipeline.
edge	Local scoring	HTTP	Baseline edge pipeline without cloud sync.
cloud_simulated_secure	Full central processing	HTTP + simulated security stages	Explain theoretical security overhead.
edge_simulated_secure	Local scoring	HTTP + simulated security stages	Explain edge-side security overhead.
cloud_tls	Full central processing	HTTPS/mTLS	Measure real transport security overhead.
edge_tls	Local scoring	HTTPS/mTLS	Measure TLS on the client-edge request.

Hybrid Google Cloud Run scenarios

Scenario	Cloud target	Edge target	What is measured
cloud	Google Cloud Run <code>/process</code>	Not used	Real client-to-Google-Cloud latency plus cloud processing.
edge	Not used	Local edge-api <code>/process</code>	Local edge processing without cloud synchronization.
cloud_simulated_secure	Google Cloud Run <code>/process</code>	Not used	Google Cloud path plus simulated auth/crypto/replay costs.
edge_simulated_secure	Not used	Local edge-api <code>/process</code>	Local edge path plus simulated security costs.

In hybrid mode, BENCHMARK_SCENARIOS is set to cloud, edge, cloud_simulated_secure, edge_simulated_secure because Cloud Run provides platform HTTPS but does not terminate the repository's local mTLS certificates inside the container. The local TLS scenarios remain available in the standard Docker Compose benchmark.

8. Benchmark Outputs

```
results/results.csv
results/summary.json
results/patient_results.json
```

`results.csv` is the technical benchmark table. `summary.json` aggregates latency, payload, and alert counts by scenario. `patient_results.json` is optimized for the hospital dashboard.

Field	Meaning
patient_id, ward, bed, diagnosis	Clinical context of the processed patient.
client_total_ms	End-to-end latency seen by the benchmark client.
service_total_ms	Time measured inside the API handler.
client_service_delta_ms	Client-side overhead not measured inside the service. TLS handshake and HTTP overhead mainly appear here.
network_ms	Configured network delay stage.
preprocess_ms	Validation, cleaning, filtering, and feature preparation stage.
inference_ms	Clinical risk scoring stage.
sync_ms	Cloud synchronization time. It is 0 for edge scenarios in the direct comparison.
payload_sent_kb	Input payload sent by the benchmark client.
payload_synced_kb	Payload sent to cloud. Full record in cloud mode, 0 in edge mode.
risk_score, risk_level	Computed patient risk.
recommended_action	Operational recommendation for the ward team.

9. Hospital Dashboard

The hospital dashboard is served by `services/hospital_dashboard/app/main.py` at `http://localhost:8080`.

- Patient worklist sorted by risk.
- Ward and risk filters.
- Latest vital signs for the selected patient.
- Diagnosis, medications, comorbidities, risk score, and recommended action.
- Per-patient comparison of cloud, edge, simulated security, TLS, and Google Cloud hybrid scenarios when the corresponding benchmark run has been executed.
- Ward load and active alert count.

The dashboard reads `patient_results.json` after the benchmark runs. If the benchmark has not been executed yet, it still displays the dataset and marks patients as pending.

10. Prometheus and Grafana

Prometheus collects runtime metrics from the API services. It does not execute the benchmark. Grafana visualizes those metrics over time. The hospital dashboard is separate: it visualizes clinical benchmark results.

```
pipeline_requests_total
pipeline_total_latency_ms
pipeline_stage_latency_ms
pipeline_payload_size_kb
pipeline_in_flight_requests
```

In the local benchmark, Prometheus scrapes Docker service names such as `cloud-api`, `edge-api`, `cloud-api-tls`, and `edge-api-tls`. In the Google Cloud hybrid benchmark, Prometheus uses `prometheus.gcp.yml` to scrape the local `edge-api` and the Cloud Run HTTPS host generated in `.env.gcp`.

11. Commands

```
docker compose up --build
docker compose run --rm benchmark

powershell -ExecutionPolicy Bypass -File scripts\deploy-cloud-run.ps1 `
  -ProjectId benchmark-edge-cloud `
  -Region europe-west8

docker compose --env-file .env.gcp `
  -f docker-compose.yml `
  -f docker-compose.gcp.yml up --build

docker compose --env-file .env.gcp `
  -f docker-compose.yml `
  -f docker-compose.gcp.yml run --rm benchmark

docker compose ps
docker compose logs -f cloud-api edge-api cloud-api-tls `
  edge-api-tls hospital-dashboard
docker compose down
```

12. Limitations

- The dataset is synthetic and small by design.
- The scoring model is deterministic and explanatory; it is not a validated medical model.
- The edge is emulated through Docker CPU/RAM limits, not real bedside hardware.
- TLS certificates are local self-signed certificates for benchmarking, not production certificates.
- Cloud Run platform HTTPS is real, but it is terminated by Google Cloud rather than by the application container.
- The hybrid Google Cloud benchmark depends on the local network path from the workstation to Google Cloud Europe.
- Network delay is simulated at application level; a stronger network study could use tc netem, Mininet, or Containernet.